

Manual do Projeto

O que cada arquivo executa, na mesma árvore de pastas do repositório. Nome do arquivo
linka direto pro código no GitHub (`annaotoni/LLM---project` , branch `main`).
Comandos de execução e teste no final.

ÍNDICE	
01 Raiz do projeto	06 app/adapters/
02 app/api/	07 scripts/
03 app/config/	08 data/sample_docs/
04 app/domain/	09 tests/
05 app/orchestration/	10 Comandos

 **Como usar.** Cada seção abaixo é uma pasta real do projeto. Dentro dela, cada linha é um arquivo com uma frase dizendo o que ele executa — não o que ele é. Clique no nome do arquivo pra abrir o código correspondente no GitHub.

/ (raiz)

Configuração de ambiente, orquestração e documentação — nada de regra de negócio aqui.

README.md	Documentação principal: fluxo, como rodar, como testar e as decisões de arquitetura resumidas.
Dockerfile	Builda a imagem do serviço <code>app</code> : <code>pip install .</code> a partir de <code>pyproject.toml</code> , expõe a porta 8000 e sobe <code>uvicorn</code> .
docker-compose.yml	Orquestra <code>postgres</code> (pgvector/pg16, com healthcheck), <code>ollama</code> e <code>app</code> — <code>app</code> só sobe depois que o Postgres fica saudável.
pyproject.toml	Dependências (FastAPI, asyncpg, litellm, langgraph, slowapi), extra <code>[dev]</code> de teste/lint e <code>testpaths</code> do pytest (só <code>tests/unit</code> e <code>tests/feature</code>).
.env.example	Modelo das variáveis: <code>DATABASE_URL</code> , <code>provider/modelo LLM</code> , modelo de embedding, URL do Ollama e <code>RAG_TOP_K</code> — com comentários sobre a diferença host vs. Docker.
github/workflows/ci.yml	CI: instala <code>[.dev]</code> , roda <code>ruff check .</code> e <code>pytest tests/unit tests/feature -v</code> , <code>tests/evals/</code> fica de fora — precisa de Postgres+Ollama reais.

app/api/

Camada HTTP — rotas finais, sem regra de negócio. Ponto de entrada da aplicação.

main.py	Monta a <code>FastAPI</code> : no <code>lifespan</code> cria o pool <code>asyncpg</code> (<code>min_size=0</code> — a API sobe mesmo com o banco fora do ar) e o fecha ao encerrar; registra o limiter do slowapi e o handler de <code>RateLimitExceeded</code> ; inclui os routers <code>health</code> e <code>chat</code> .
dependencies.py	Executa <code>obter_tenant_id</code> : lê <code>Authorization: Bearer <chave></code> , autentica contra <code>TenantRepositoryInterface</code> (hash da chave, nunca texto puro); sem header ou chave inválida, levanta <code>401</code> . Grava <code>tenant_id</code> e <code>modelo_override</code> em <code>request.state</code> , de onde o rate limiter e a rota de chat leem sem repetir a consulta.
limiter.py	Instancia o <code>Limiter</code> do slowapi com <code>key_func=obter_chave_limite</code> : cota por <code>"{ip}:{tenant_id}"</code> — lê o tenant já autenticado em <code>request.state</code> , não consulta o banco de novo.
routes/chat.py	Define <code>POST /chat</code> (<code>20/minute</code> por IP+tenant): monta o contexto, cria as duas ferramentas já com <code>tenant_id</code> fixado, roda o grafo e reaproveita <code>resposta_final</code> se o grafo já fechou a resposta — só chama <code>stream_completion</code> de novo no corte de <code>MAX_ITERACOES</code> . Transmite em SSE, mascarando PII pedaço a pedaço.
routes/health.py	Define <code>GET /health</code> (liveness pura) e <code>GET /ready</code> (abre uma conexão <code>asyncpg</code> direta com timeout de 2s; falha vira <code>503</code>).

app/config/

Configuração tipada e logging — atravessa todas as camadas.

settings.py	Define <code>Settings</code> (pydantic-settings) a partir do <code>.env</code> : <code>database_url</code> obrigatório, <code>provider/modelo LLM</code> , modelo de embedding e <code>rag_top_k</code> . <code>get_settings()</code> é cacheado com <code>@lru_cache</code> — singleton por processo.
logging.py	Define <code>FormatadorJSON</code> , que serializa cada log (nível, logger, mensagem + campos <code>extra</code>) como JSON; <code>configurar_logging()</code> instala esse formatter no stdout.
app/domain/	

Regra pura, sem I/O. Nada aqui importa FastAPI, asyncpg, litellm ou LangGraph.

dto/chat.py	Define <code>MensagemChatEntrada</code> (Pydantic): <code>mensagem</code> entre 1 e 4000 caracteres, com validador que dá <code>strip()</code> e rejeita mensagem vazia/só espaço.
entities/documento.py	Dataclass congelada <code>DocumentoRecuperado</code> (<code>titulo</code> , <code>conteudo</code> , <code>distancia</code>) — resultado da busca vetorial.
entities/pedido.py	Dataclass congelada <code>Pedido</code> (<code>numero_pedido</code> , <code>status</code> , <code>previsao_entrega</code> , <code>cliente_nome</code>).
entities/tenant.py	Dataclass congelada <code>TenantAutenticado</code> (<code>tenant_id</code> , <code>modelo_override</code>) — retorno da autenticação por chave de API.
guardrails/pii.py	Executa <code>mascarar_pii</code> (regex de CPF, e-mail, cartão e telefone → <code>[X OCULTADO]</code>) — CPF/cartão/telefone sem <code>\b</code> , que não marca fronteira entre letra e dígito) e <code>mascarar_stream_pii</code> , que só libera texto até o último espaço em branco do buffer acumulado — nenhum padrão fica cortado ao meio entre dois pedaços do stream.
guardrails/sanitizacao.py	Executa <code>sanitarizar_conteudo_externo</code> : trunca o texto recuperado em 2000 caracteres e substitui marcadores de instrução embutidos (<code>system</code> : , <code>###</code> , "ignore as instruções anteriores") por <code>[trecho removido]</code> .

app/orchestration/

Gateway de modelo, agente LangGraph, contexto e prompts versionados.

gateway/model_gateway.py	Executa <code>ModelGateway</code> — único ponto de contato com provedores de LLM via LiteLLM: <code>complete_with_tools</code> (sem streaming, para o agente ler <code>tool_calls</code>), <code>stream_completion</code> (streaming, resposta final) e <code>embed</code> . Os dois primeiros aceitam um <code>modelo</code> opcional — usado no lugar do <code>LLM_MODEL</code> global quando o tenant tem <code>modelo_override</code> . Toda chamada marca <code>metadata={ "tenant_id" , "prompt_id" }</code> pro LiteLLM e loga tenant, prompt, modelo usado, latência, custo (<code>litellm.completion_cost</code> , best-effort) e tokens.
agent/graph.py	Compila o <code>StateGraph</code> do agente (nós <code>modelo</code> ⇄ ferramentas) uma única vez por processo. Quando o modelo não pede mais ferramenta, guarda o texto já gerado em <code>resposta_final</code> — evita que <code>chat.py</code> peça a mesma resposta de novo. <code>decidir_proximo_passo</code> encerra em <code>END</code> quando o modelo não pede ferramenta ou quando <code>iteracoes > MAX_ITERACOES</code> (4).
context/builder.py	Executa <code>construir_contexto</code> : monta <code>[system, user]</code> do zero a cada chamada (o modelo não tem memória própria) a partir do prompt versionado, retornando também o <code>prompt_id</code> .
prompts/chat_agente_v1.yaml	Prompt versionado do agente (<code>id</code> : <code>chat.agente</code> , <code>version</code> : <code>1</code>): instruções em português descrevendo as duas ferramentas disponíveis e a regra de nunca inventar informação.
prompts/loader.py	Executa <code>carregar_prompt</code> (<code>@lru_cache</code>): lê o YAML do prompt e retorna um <code>Prompt(id, version, template)</code> — cache em memória por processo.

app/adapters/

Implementa as portas com tecnologia real (Postgres) e hospeda as ferramentas do agente.

database.py	Executa <code>obter_pool</code> : expõe o pool <code>asyncpg</code> já criado no <code>lifespan</code> (via <code>request.app.state.pool</code>) para injeção por <code>Depends</code> .
repositories/contracts/documento_repository_interface.py	<code>Protocol DocumentoRepositoryInterface</code> : <code>buscar_similares</code> e <code>inserir</code> — contrato puro, sem implementação.
repositories/contracts/pedido_repository_interface.py	<code>Protocol PedidoRepositoryInterface</code> : <code>buscar_por_numero</code> (<code>tenant_id</code> , <code>numero_pedido</code>) .
repositories/contracts/tenant_repository_interface.py	<code>Protocol TenantRepositoryInterface</code> : <code>autenticar_por_chave</code> (<code>chave_api</code>) -> <code>TenantAutenticado</code> <code>None</code> .
repositories/postgres/documento_repository.py	Implementa a busca por similaridade com o operador pgvector <code><=></code> (distância de cosseno) filtrando por <code>tenant_id</code> , e o <code>upsert</code> (<code>ON CONFLICT (tenant_id, titulo)</code>) na ingestão. <code>_vetor_para_sql</code> rejeita embedding com dimensão diferente de <code>DIMENSAO_EMBEDDING</code> (384) antes de montar o SQL.
repositories/postgres/pedido_repository.py	Implementa <code>buscar_por_numero</code> : <code>SELECT</code> filtrado por <code>tenant_id</code> + <code>numero_pedido</code> , retorna <code>None</code> se não encontrar.
repositories/postgres/tenant_repository.py	Executa <code>hash_chave_api</code> (SHA-256) e <code>TenantRepositoryPostgres.autenticar_por_chave</code> : consulta <code>tenant_id</code> e <code>modelo_override</code> por <code>chave_api_hash</code> — a chave em texto puro nunca chega ao banco nem é comparada diretamente.
tools/ferramenta.py	Dataclass genérica <code>Ferramenta</code> (<code>executar</code>) — contêiner que o grafo usa para representar qualquer tool.
tools/buscar_documentos.py	Cria a ferramenta <code>buscar_documentos</code> : gera embedding da consulta, busca no repositório já filtrado por <code>tenant_id</code> (fixado por closure) e sanitiza o texto antes de devolver ao agente.
tools/consultar_pedido.py	Cria a ferramenta <code>consultar_pedido</code> : busca o pedido isolado por <code>tenant_id</code> e formata status + previsão de entrega (ou "não encontrado").

scripts/

Ingestão de documentos e seed de pedidos/tenants — rodados uma vez, fora do ciclo de request.

ingest_documents.py	Percorre <code>data/sample_docs/<tenant>/*.md</code> (nome da pasta = <code>tenant_id</code>), gera embedding de cada documento via <code>ModelGateway.embed</code> e faz upsert no Postgres.
seed_pedidos.py	Insere/atualiza uma lista fixa de 5 pedidos de exemplo (3 <code>loja-azul</code> , 2 <code>loja-verde</code>) na tabela <code>pedidos</code> .
seed_tenants.py	Insere/atualiza a chave de API de cada loja de exemplo na tabela <code>tenants</code> — grava só o hash (<code>hash_chave_api</code>), nunca a chave em texto puro.
schema.sql	Cria a extensão <code>vector</code> e as tabelas <code>documentos</code> (embedding VECTOR(384) , índice HNSW por cosseno), <code>pedidos</code> e <code>tenants</code> (<code>chave_api_hash</code> único, <code>modelo_override</code> nullable) — as três com <code>UNIQUE</code> /índice por tenant.

data/sample_docs/

Documentos de exemplo, um diretório por tenant — o conteúdo ingerido pelo RAG.

loja-azul/ <code>loja-azul</code>	<code>devolucao.md</code> , <code>frete.md</code> , <code>garantia.md</code> , <code>pagamento.md</code> — devolução em 7 dias corridos, reembolso em até 10 dias úteis.
loja-verde/ <code>loja-verde</code>	Mesmos quatro temas, política divergente: devolução em 30 dias corridos — confirma que a segmentação por tenant produz conteúdo realmente distinto, não só rótulo.

tests/

Unitário e feature não dependem de Postgres/Ollama reais — tudo mockado. `evals/` é a exceção.

conftest.py	Fixture <code>autouse</code> : define <code>DATABASE_URL</code> fake para todo teste sem o marker <code>eval</code> (que usa o <code>.env</code> real) e limpa o cache de <code>get_settings</code> antes/depois.
unit/test_context_builder.py	Confirma a ordem <code>[system, user]</code> e o <code>prompt_id == "chat.agente@v1"</code> .
unit/test_dto_chat.py	Confirma que <code>MensagemChatEntrada</code> rejeita mensagem só com espaços e faz <code>strip()</code> nas bordas.
unit/test_graph.py	Testa o <code>StateGraph</code> com um gateway falso: encerra sem tool call (e confirma que <code>resposta_final</code> é preenchida), executa uma tool fake e integra o resultado, e corta em <code>MAX_ITERACOES</code> sem <code>resposta_final</code> quando o modelo insiste em chamar ferramenta.
unit/test_limiter.py	Testa <code>obter_chave_limite</code> com um <code>Request</code> real (sem TestClient): combina IP+tenant, separa parâmetros diferentes no mesmo IP, separa o mesmo tenant em IPs diferentes, e cai no valor padrão sem tenant resolvido.
unit/test_model_gateway.py	Testa <code>_api_base()</code> (só preenchido para Ollama), que <code>stream_completion</code> passa <code>_api_base</code> apenas nesse caso, a <code>metadata</code> com tenant/prompt em toda chamada, e que <code>modelo</code> substitui o <code>LLM_MODEL</code> global quando informado (em <code>stream_completion</code> e <code>complete_with_tools</code>).
unit/test_pii.py	Testa <code>mascarar_pii</code> (CPF/e-mail/telefone, incluindo PII colada a uma letra sem espaço) e <code>mascarar_stream_pii</code> com um e-mail dividido entre dois pedaços consecutivos do stream.
unit/test_prompt_loader.py	Confirma <code>id</code> , <code>version</code> e conteúdo do template carregado.
unit/test_repositories.py	Testa os três repositórios Postgres com pool mockado: formatação do vetor e rejeição de dimensão incompatível, parâmetros passados a <code>fetch / execute / fetchrow</code> , casos vazio/não encontrado, que a busca por tenant é sempre pelo hash da chave — nunca pela chave em texto puro — e que <code>modelo_override</code> vem junto na mesma consulta.
unit/test_sanitizacao.py	Testa <code>sanitarizar_conteudo_externo</code> : remove marcador de injeção, preserva conteúdo legítimo, trunca em 2000 caracteres.
unit/test_tools.py	Testa as duas factories de ferramentas com repositórios/gateway falsos: isolamento por tenant, formatação de resultado, mensagens de erro e "não encontrado".

feature/test_chat.py	Via <code>TestClient</code> com dependências trocadas por duplês: 401 sem <code>Authorization</code> ou com chave inválida, 200 com chave válida (reaproveitando <code>resposta_final</code> ou recorrendo a <code>stream_completion</code> no corte de iterações, usando o <code>modelo_override</code> do tenant quando configurado), 422 para mensagem vazia, confirma que estourar a cota de um tenant não bloqueia outro no mesmo IP, e que o log mascara PII.
feature/test_health.py	Testa <code>/health</code> (sempre ok) e <code>/ready</code> mockando <code>asyncpg.connect</code> para simular banco fora (503) e disponível (200).
evals/dataset_rag.py	Lista <code>CASOS</code> : 10 pares pergunta → título esperado (5 por tenant), cobrindo devolução, frete, garantia e pagamento.
evals/test_eval_rag.py	Marcado <code>@pytest.mark.eval</code> — roda os 10 casos contra o Postgres+pgvector real, mede recall@3 e falha se cair abaixo de 0.7.

Comandos

O que cada comando executa, na ordem em que você provavelmente vai usá-los.

SEM DOCKER — PYTEST, TUDO MOCKADO
<code>\$ pip install -e ".[dev]"</code>
Instala o projeto em modo editável com as dependências de dev (pytest, pytest-asyncio, httpx, ruff).
<code>\$ pytest</code>
Roda só <code>tests/unit</code> e <code>tests/feature</code> — é o que <code>testpaths</code> no <code>pyproject.toml</code> declara. Tudo mockado (fakes em memória, <code>TestClient</code>), nunca toca Postgres ou Ollama de verdade.
<code>\$ pytest tests/unit</code>
Só a regra de negócio isolada: domínio, agente, gateway, ferramentas e repositórios (com pool mockado).
<code>\$ pytest tests/feature</code>
Só os endpoints via <code>TestClient</code> : <code>/health</code> , <code>/ready</code> e <code>/chat</code> com dependências trocadas por duplês.
<code>\$ pytest tests/evals -v</code>
Fora de <code>testpaths</code> — precisa ser chamado com o caminho explícito. Exige Postgres+pgvector e Ollama reais, com os documentos já ingeridos (<code>ingest_documents.py</code>).
<code>\$ ruff check .</code>
Lint — é o mesmo comando que o CI roda antes dos testes.

COM DOCKER — STACK COMPLETO

<code>\$ cp .env.example .env</code>
Copia o modelo de variáveis pro arquivo que o compose e a aplicação de fato leem.
<code>\$ docker compose up -d postgres ollama</code>
Sobe só a infraestrutura (sem a API) — útil pra rodar ingestão/uvicorn a partir do host durante o desenvolvimento.
<code>\$ docker exec -it \$(docker compose ps -q ollama) ollama pull qwen2.5:3b</code> <code>\$ docker exec -it \$(docker compose ps -q ollama) ollama pull all-minilm</code>
Baixa, dentro do container Ollama, o modelo de chat (~1,9 GB) e o de embedding (~45 MB) — só necessário na primeira vez; ficam no volume <code>ollama_data</code> .
<code>\$ python scripts/ingest_documents.py</code> <code>\$ python scripts/seed_pedidos.py</code> <code>\$ python scripts/seed_tenants.py</code>
Ingerem os documentos de exemplo (com embedding real), os pedidos de exemplo e a chave de API (só o hash) dos dois tenants — idempotentes via upsert.
<code>\$ uvicorn app.api.main:app --reload --port 8001</code>
Sobe a API fora do container, pra dev — <code>--port 8001</code> porque a 8000 costuma estar ocupada por outros projetos locais.
<code>\$ docker compose up --build</code>
Builda a imagem e sobe os três serviços juntos — alternativa a rodar <code>uvicorn</code> manualmente fora do container.
<code>\$ docker compose down</code>
Para e remove os containers da stack ao final.

EXERCITANDO A API

<code>\$ curl http://localhost:8001/ready</code>
Confirma que a API está de pé e consegue abrir conexão com o Postgres. Retorna <code>{ "status": "ready" }</code> . Esse GET simples funciona igual em bash e no <code>curl / Invoke-WebRequest</code> do PowerShell.

<code>\$ curl -N -X POST http://localhost:8001/chat \</code> <code>-H "Content-Type: application/json" -H "Authorization: Bearer dev-loja-azul" \</code> <code>-d '{"mensagem": "Qual o prazo para devolução de um produto?"}'</code>
Em bash/WSL, <code>-N</code> desativa o buffer do curl, necessário pra ver o SSE chegando pedaço a pedaço em vez de tudo de uma vez no final. <code>dev-loja-azul</code> é a chave semeada por <code>seed_tenants.py</code> ; sem <code>Authorization</code> ou com chave errada: 401 .

PS> \$pergunta = "Qual o prazo para devolução de um produto?"
PS> \$json = @{ mensagem = \$pergunta } | ConvertTo-Json
PS> \$bytes = [System.Text.Encoding]::UTF8.GetBytes(\$json)
PS> Invoke-RestMethod -Uri http://localhost:8001/chat -Method Post ` ` -Body \$bytes -ContentType "application/json; charset=utf-8" ` -Headers @{ "Authorization" = "Bearer dev-loja-azul" } `

Equivalente no PowerShell, `curl` ali é apelido de `Invoke-WebRequest` — não aceita `-N` / `-H` / `-d` , e a barra `\` não continua linha (é `Invoke-RestMethod` + backtick ``` que fazem esse papel). Sem streaming incremental: só devolve tudo quando o SSE terminar.

 Cuidado com acento no PowerShell Desktop 5.1. `Invoke-RestMethod -Body $json` (passando a *string* direto) codifica pelo codepage do console em vez de UTF-8 — um acento como em "devolução" vira bytes inválidos como UTF-8 e o FastAPI responde 400 There was an error parsing the body . `[System.Text.Encoding]::UTF8.GetBytes(...)` explícito (comando acima) evita o problema. Mesma família de causa do encoding discutido no [Dashboard de Execução](#) .

<code>\$ start http://localhost:8001/docs</code>
Abre o Swagger gerado pelo FastAPI — mas <code>/chat</code> é streaming SSE, então a UI do Swagger mostra o schema, não é o melhor lugar pra testar a resposta pedaço a pedaço (use os exemplos acima).

Referência de navegação do repositório — não substitui o README, só facilita achar onde cada coisa está.